Intent Compression Architecture (ICA)

Clarification-first ambiguity management for higher precision, lower token waste, and more reliable agents

Paul Maddison • paul.maddison.delimeg@gmail.com • March 2026 • v5.0 (detailed)

Executive Summary

Large language models often answer the literal surface form of a question even when the user's intended meaning depends on unstated definitions, missing constraints, or implicit assumptions. When a query has multiple plausible interpretations, the model effectively spreads probability across those interpretations. The resulting response frequently becomes a "mixture answer": longer, broader, and hedged to remain defensible across competing meanings.

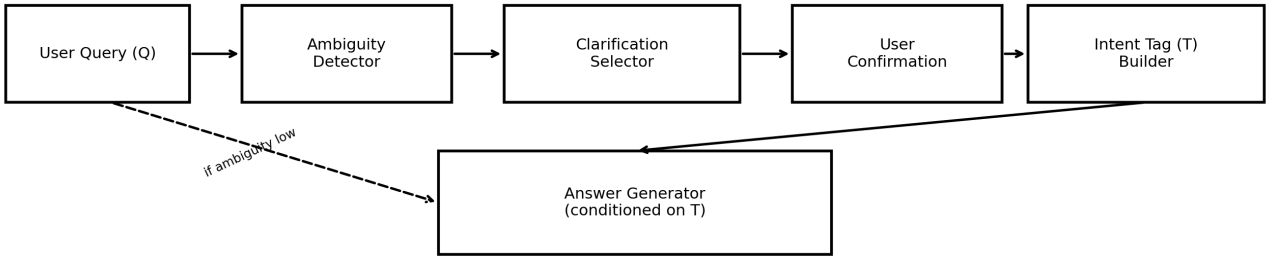
This is not primarily a tone problem. It is a structural ambiguity problem. Ambiguity increases interpretive uncertainty, which increases answer breadth. Answer breadth increases token consumption and reduces specificity. In agent workflows (coding agents, research agents, tool-using planners), early ambiguity increases branching, retries, and loop depth, turning a small missing constraint into exponential compute waste.

Intent Compression Architecture (ICA) is a clarification-first structural layer that collapses interpretive uncertainty before full answer generation. If the ambiguity is low, the system answers immediately (no extra friction). If the ambiguity is high and materially changes the correct answer, the system asks one minimal, targeted clarification question designed to remove the largest amount of uncertainty per token. Once the user confirms the intended meaning, the final response is generated conditioned on an explicit intent tag.

ICA changes generation from P(A | Q) to P(A | Q, T), where Q is the original query and T is the confirmed intent representation. This narrows the output distribution, reduces unnecessary hedging, and improves precision. It also reduces tokens per resolved task for many ambiguous prompts, because a short clarification plus a narrower answer is cheaper than a long hedged answer.

At scale, ICA produces a stream of structured intent-labelled interactions (Q, C, T) where C is the clarification asked and T is the user-confirmed intent. This dataset is higher-signal than preference-only feedback because it supervises what the user meant, not just which answer sounded acceptable. As this data accumulates, ambiguity detection and intent prediction improve, the system triggers fewer clarifications while keeping precision high, and the overall token efficiency improves over time (a compounding effect).

High-level pipeline


User Query (Q)
Ambiguity Detector
Clarification Selector
User Confirmation
Intent Tag (T) Builder
if ambiguity low
Answer Generator (conditioned on T)

1. Background: why this happens in real systems

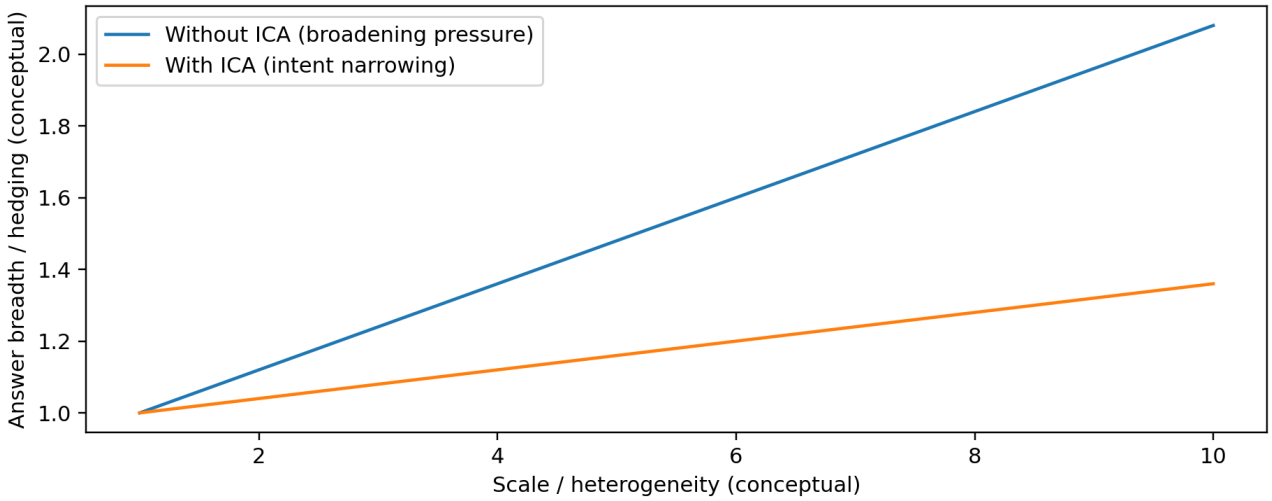
People ask questions in shorthand. They expect listeners to fill in definitions and assumptions. Models cannot reliably infer unstated intent, so ambiguous queries induce multiple plausible meanings. In practice, models often respond with cautious multi-perspective answers because (a) the term is contested, (b) committing to one meaning risks being “wrong” under another meaning, and (c) preference optimisation can reward broadly acceptable phrasing in ambiguous cases.

This produces a predictable failure mode: the user asked a question with an implied definition; the model answered a different question (the literal or most defensible interpretation). The mismatch looks like evasiveness or bias, but it is often an intent-resolution failure.

2. Reverse Funnel hypothesis (scale → broader answers)

Reverse Funnel is the hypothesis that at large user scale, ambiguous prompts increasingly receive broad, generalised answers because interpretive variance increases with heterogeneity, and optimisation pressure favours safety/acceptability across interpretations. The “funnel” is reversed: instead of narrowing to a single meaning, the model expands coverage to remain acceptable.

Conceptually, if interpretive entropy H(I|Q) increases with scale/heterogeneity, then answer breadth tends to increase unless the system has a mechanism to collapse uncertainty before answering. ICA is that mechanism.



3. Formal framing: interpretations, entropy, and conditioned answers

Let Q be a query. Let I be a latent variable representing the intended interpretation. Q induces a distribution P(I|Q). Interpretive entropy is:

```
H(I|Q) = − Σ P(I■|Q) log P(I■|Q)
```

Standard answering implicitly integrates across interpretations: P(A|Q) = Σ P(A|I■,Q)P(I■|Q). ICA introduces a confirmation step yielding intent tag T so that generation becomes P(A|Q,T). Conditioning on T collapses the mixture and narrows the output distribution.

ICA is not “asking more questions for the sake of it”. The aim is to minimise total cost by spending a small number of tokens to reduce entropy early, then producing a narrower answer.

4. Ambiguity taxonomy (what triggers clarification)

Type	Description
Definition ambiguity	Contested/loaded labels where the definition determines the answer (e.g., “propaganda”, “misinforma
Constraint ambiguity	Missing parameters that change output (time range, locale, environment, budget, input/output format,
Referential ambiguity	Unclear entities or pronouns (“this”, “they”, multiple candidates).
Causal ambiguity	Causal claims need a threshold/counterfactual baseline (“did X cause Y?”).
Task-type ambiguity	User intent could be summary vs evaluation vs instruction vs code vs debugging.

Trigger policy: only ask when the answer would materially change under plausible interpretations. This avoids “annoying” over-clarification.

5. Clarification selection: maximum uncertainty reduction per token

ICA's clarification selector should be optimised to collapse the largest uncertainty with minimal tokens. The design objective can be written as:

```
C* = argmax_C ( H(I|Q) − H(I|Q,C) ) / TokenCost(C)
```

In practical terms, the best clarifications are often short, structured, and offer concrete choices (2–4 options) or ask for a single missing constraint. Open-ended "please clarify" prompts waste tokens and increase friction.

Examples of high-efficiency clarifications:

• "When you say 'propaganda', do you mean (A) persuasive political advocacy, (B) coordinated deceptive messaging, or (C) another definition?"

• "Which environment are you running this in: Windows, macOS, Linux, or Docker?"

• "Do you want a summary of evidence, or an evaluation/judgement under a specific definition?"

6. Token economics: why clarification can reduce compute

Let T_base be the average tokens for a hedged answer when ambiguity is present. Let T_c be tokens for a clarification and T_cond be tokens for the conditioned answer. ICA total is: T_ICA = T_c + T_cond. ICA reduces tokens when T_c + T_cond < T_base.

Illustrative worked example (numbers can be measured empirically):

```
T_base = 280, T_c = 22, T_cond = 140 ⇒ T_ICA = 162 (≈ 42% reduction)
```

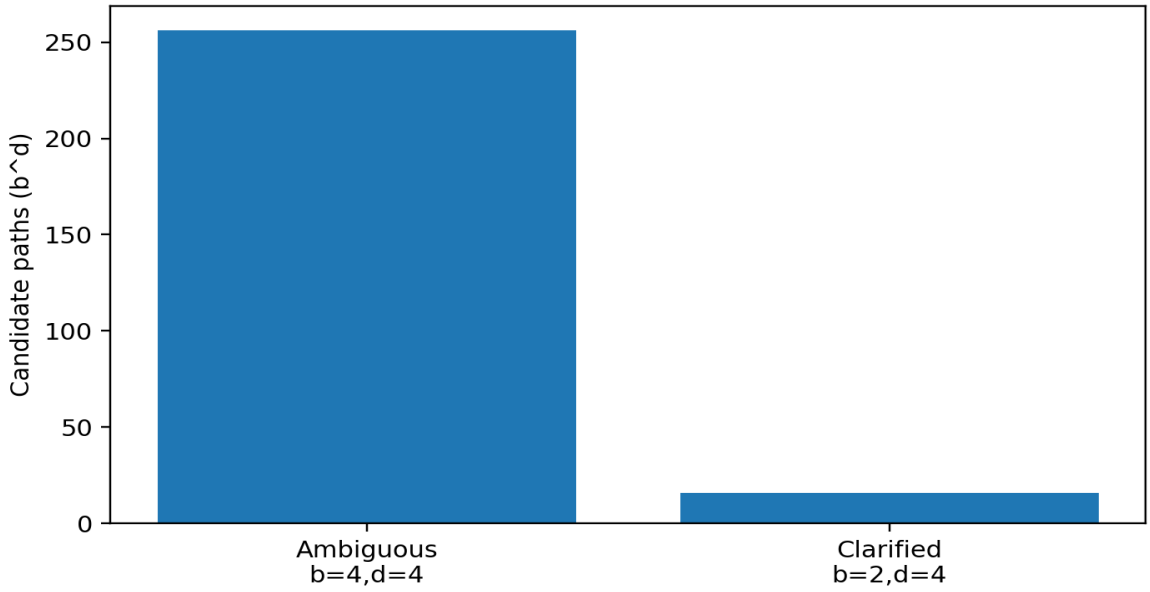
At scale, even small reductions matter. If ambiguous prompts are a fraction p of traffic and average savings per ambiguous prompt is ΔT, then average savings per query is $p \cdot \Delta T$.

```
Average token savings per query = p · ΔT
```

This is testable in A/B experiments (see Section 10).

7. Agent systems: why early ambiguity explodes cost

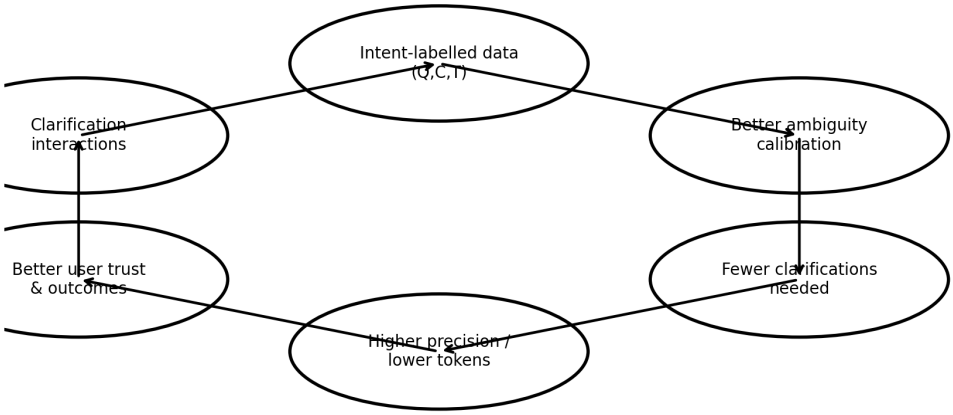
Agentic workflows (coding agents, research agents, tool-using planners) plan over sequences of actions. Ambiguity early in the plan increases the branching factor b (more possible next steps), which increases cost roughly as O(b^d) for depth d. Confirming the missing constraint early can cut b and reduce paths exponentially.



This is why ICA is especially valuable for agents: the clarification happens before the expensive loop begins.

8. Scale flywheel: why ICA improves over time

Each clarification produces structured intent supervision: (Q, C, T). This is high-signal data about what the user meant. Over time, this can improve: (1) ambiguity detection calibration, (2) intent prediction (so the system can infer T without asking), and (3) clarification selection quality. As these improve, the system triggers fewer clarifications while preserving precision and saving tokens.



9. Interaction with preference optimisation (RLHF-style signals)

When prompts are ambiguous, preference optimisation can favour broadly acceptable answers to avoid negative feedback. That can push the model toward generalised multi-interpretation responses. ICA changes the optimisation landscape by turning ambiguous prompts into clarified prompts. Preference feedback is then attached to (Q,T) rather than only Q, reducing generalisation pressure and improving learning signal quality.

This is not a claim about any specific training method; it is a generic observation: clarifying intent reduces the space of “acceptable” answers because the question becomes well-defined.

10. Evaluation plan (what to measure and how)

ICA should be evaluated with (a) online A/B tests and (b) controlled benchmarks containing ambiguous prompts. The key is to measure end-to-end outcomes, not just subjective preference.

Metric	Definition	Why it matters
Total Tokens per Resolved Task (TTRT)	All tokens used until the user's goal is achieved, including clarifications.	Compute cost + UX latency proxy.
Clarification Trigger Rate	How often ICA asks a clarification.	User friction; should decline over time.
Clarification Success Rate	Fraction of clarifications that lead to a clearer, more direct response.	Measures whether clarifications are useful.
Response Specificity Index (RSI)	Lower hedging density; higher constraint specificity after confirmation.	Precision and perceived quality.
Agent Loop Depth Reduction (ALDR)	Average number of iterations/retries in agent workflows.	Reliability for coding/research agents.
Success Rate @ Budget	Task success under fixed token/compute budgets.	Operational efficiency.

Suggested experiment design: route a random fraction of traffic through ICA for ambiguity-likely prompts, then compare TTRT and success metrics against baseline. For agents, run identical task suites with and without ICA constraint-confirmation and compare loop depth and success under budget.

11. Risks, failure modes, and mitigations

Risk	What happens	Mitigation
Over-clarification friction	Asking too often harms UX.	Use calibrated thresholds; ask only when the answer ma
False-premise legitimisation	Clarifying can look like endorsing a false premise.	Premise-aware clarifications: challenge the premise or r
Adversarial manipulation	Users may try to force a preferred framing through clarifications.	Detect manipulation; constrain clarification options; log a
Latency	Extra turn adds time.	Ask short structured clarifications; cache inferred intents
Coverage gaps	Ambiguity detector misses cases or triggers incorrectly.	Continuous calibration; targeted datasets of ambiguous

12. Implementation roadmap (practical, staged rollout)

Phase 0 (measurement): add instrumentation to estimate ambiguity likelihood and token inflation on ambiguous prompts (no UX change).

Phase 1 (agents-first): enable clarification-first for coding/research agents where constraints are routinely missing and loop costs are high. Start with constraint ambiguity (environment, I/O, libraries, goals).

Phase 2 (consumer UX): enable ICA for a narrow set of ambiguity categories (definition ambiguity + task-type ambiguity) with conservative thresholds. Measure friction vs savings.

Phase 3 (training integration): store (Q,C,T) pairs and fine-tune ambiguity detection and intent prediction to reduce future clarifications.

Phase 4 (policy optimisation): optimise clarification selection for maximum entropy reduction per token; personalise thresholds; improve premise-aware handling.

13. Worked examples (before vs after)

Example A (definition ambiguity):

User: “Is X spreading propaganda?”

Baseline: long answer defining propaganda, listing multiple perspectives, hedging across definitions.

ICA clarification: “When you say ‘propaganda’, do you mean (A) persuasive political advocacy, (B) coordinated deceptive messaging, or (C) another definition?”

User: “A”

ICA answer: direct response grounded in definition A, with minimal hedging and a clear statement of the chosen definition.

Example B (coding constraint ambiguity):

User: “My build fails with SSL error. Fix it.”

ICA clarification: “Which environment are you using (OS, language/runtime, package manager), and can you paste the full error trace?”

Result: the first answer is immediately actionable, reducing retries.

14. Conclusion

ICA is a structural ambiguity-management architecture: confirm intent when it matters, then answer precisely. It reduces mixture answers, saves tokens, stabilises agents, and creates high-signal intent supervision at scale. The proposal is measurable, testable, and compatible with existing LLM deployment pipelines.

Author

Paul Maddison
paul.maddison.delimeg@gmail.com
https://www.linkedin.com/in/paul-maddison-b83395175/